

Spring, Spring Boot & Microservices

SPRING & MICROSERVICE NOTES

DAY 1

SPRING CORE

INTRODUCTION TO SPRING FRAMEWORK

- **Overview of Spring Framework**
- **Features**
- **Lightweight Framework**
- **IoC**
- **Spring Container**
- **Exception Handling**
- **Integration with other frameworks**
- **MVC**
- **Spring Modules**
- **Setting up Spring Environment**
- **Spring Containers**
- **BeanFactory**
- **ApplicationContext**
- **Hands-on on Spring Container**

SPRING & MICROSERVICE NOTES

Day-1

WHAT IS SPRING FRAMEWORK?

- **Spring Framework is an open-source Java framework developed to simplify enterprise application development.**
- **Before Spring, Java EE applications required developers to write a lot of boilerplate code. Spring reduces complexity by managing object creation, dependency injection, transactions, and many other infrastructure concerns.**

Spring Framework is a lightweight, open-source framework that provides comprehensive infrastructure support for developing Java applications.

Why was Spring introduced?



Problems before Spring:

- Too much object creation using new
- Tight coupling between classes
- Difficult unit testing
- Large XML configuration
- Complex transaction management
- Difficult database integration

Spring solves these problems using:

- IoC
- Dependency Injection
- AOP
- JDBC Template
- Transaction Management
- MVC
- Security

Features of Spring

1. Lightweight

Only the required modules are loaded.

Example

A console application can use only Spring Core.

2. Dependency Injection

Spring creates and injects objects.

Instead of:

`Student student = new Student();`

Spring creates the object.

3. IoC Container

The container manages:

- Object creation
- Dependencies
- Bean lifecycle
- Bean destruction

4. Modular Architecture

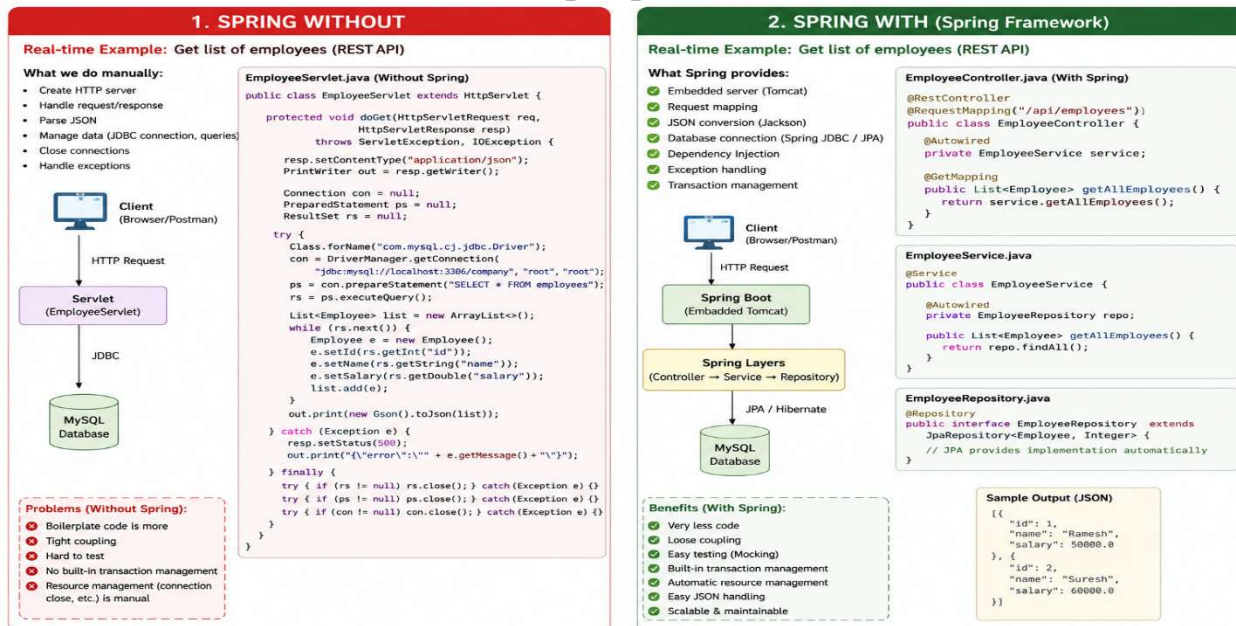
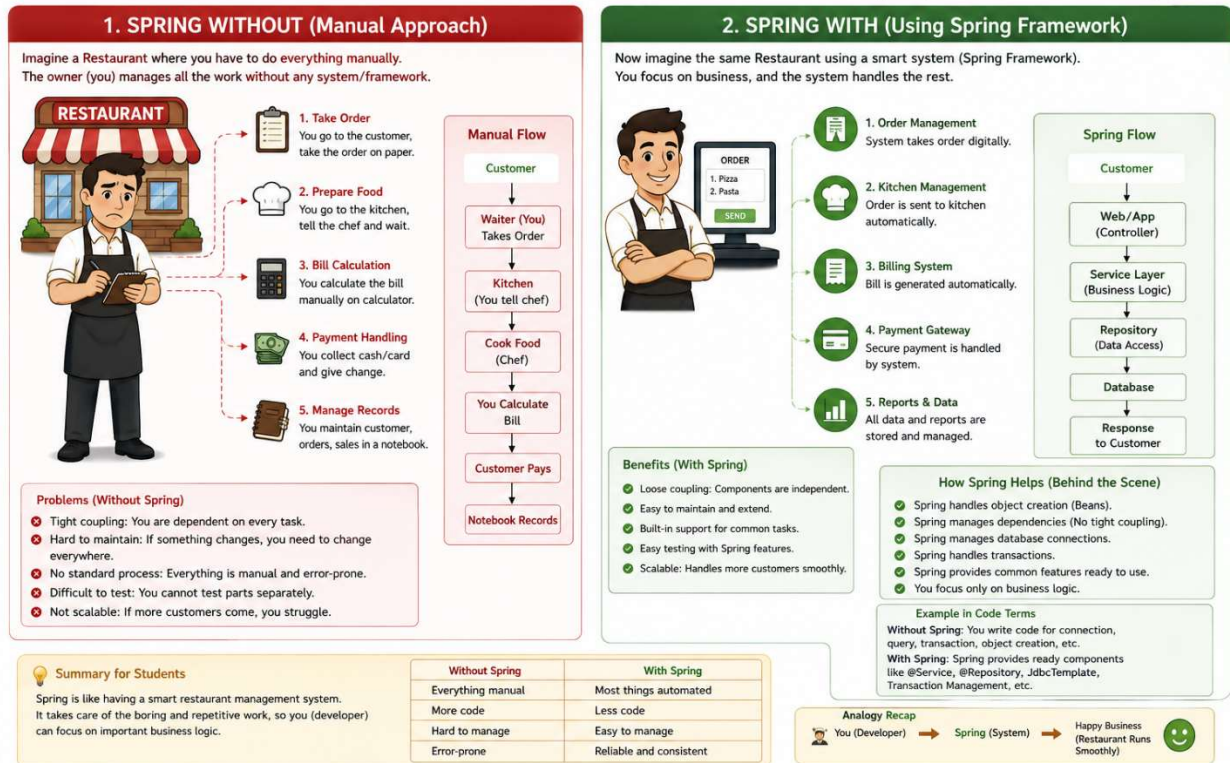
Modules include:

- Spring Core
- Spring Context
- Spring JDBC
- Spring MVC
- Spring Boot
- Spring Security
- Spring Data JPA

5. Easy Testing

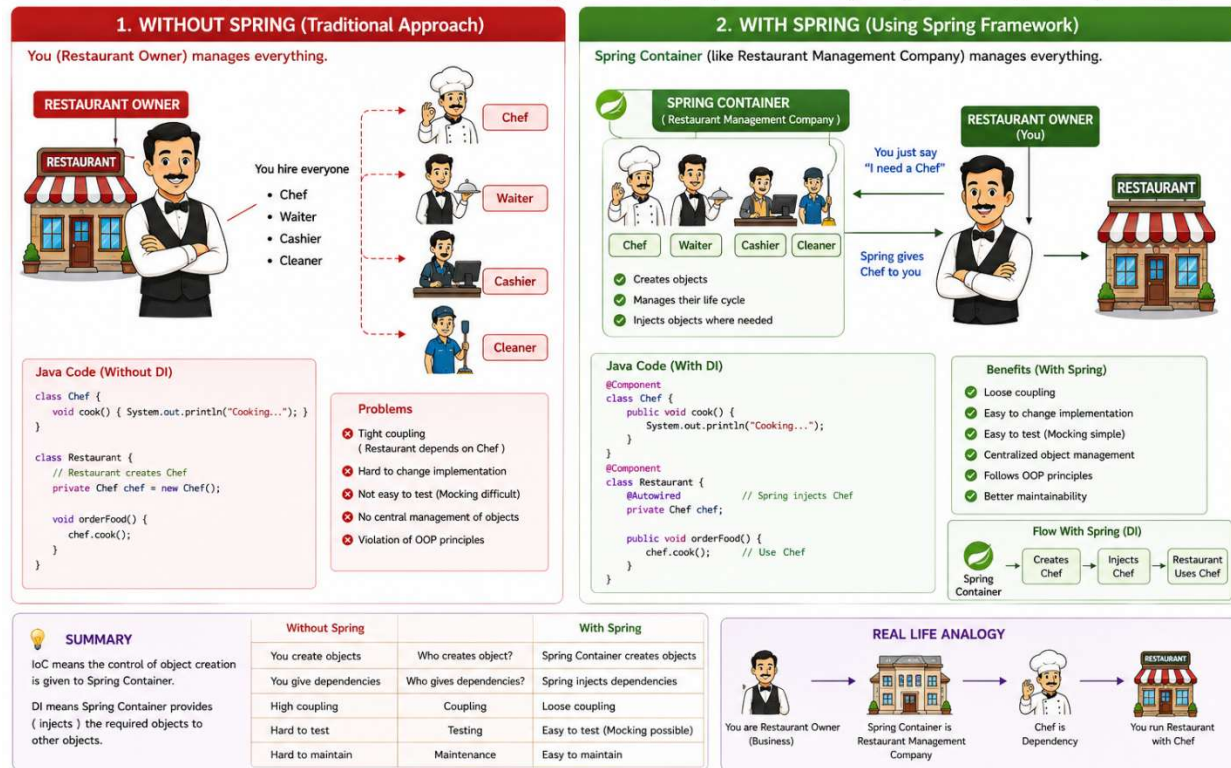
JUnit integration is straightforward because dependencies can be injected and mocked

SPRING WITHOUT vs SPRING WITH – Real Time Example (Restaurant Analogy)



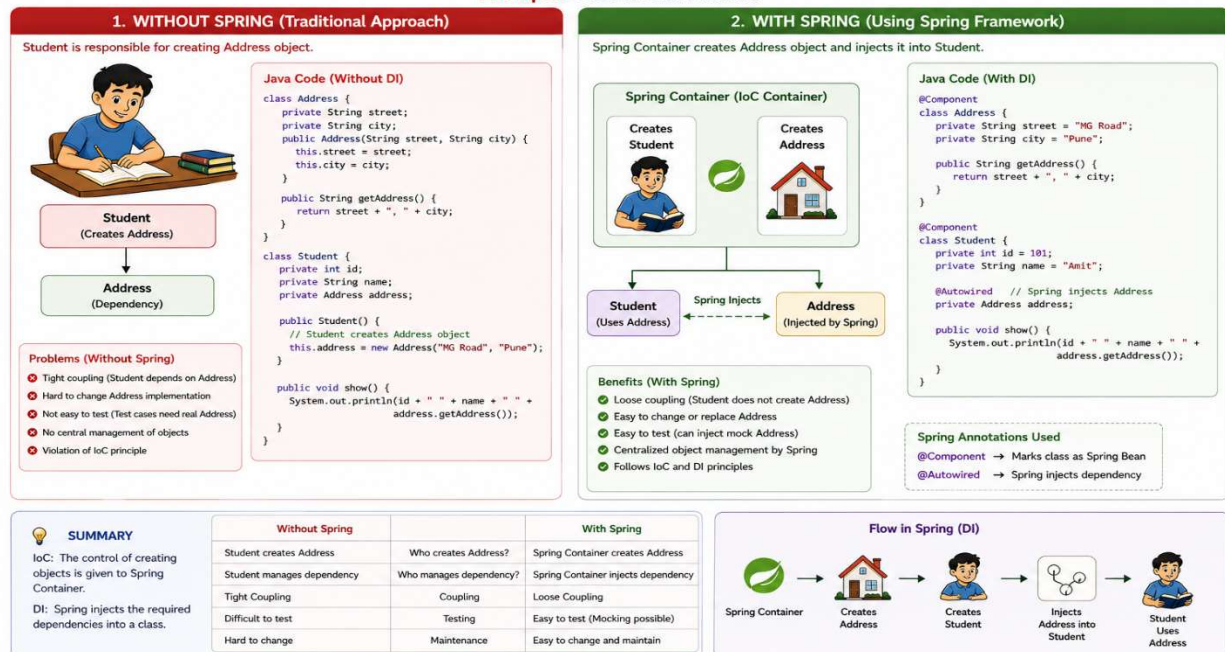
IOC

IoC (Inversion of Control) & DI (Dependency Injection) in Spring



IoC (Inversion of Control) & DI (Dependency Injection) in Spring

Example: Student and Address



IOC

- Inversion of Control is an architectural pattern describing an external entity (that is container) used to wire objects at creation time by injecting their dependencies, that is, connecting the objects so that they can work together in a system. In other words the IOC describes that a dependency by the component itself. "DI is a process of injecting (pushing) the dependencies into an object".

Why Dependency Injection (DI)?

- The most of enterprise applications now are implemented by using OOP methodology. In developing medium and huge systems using the object-oriented programming methodology.
- We divide the system into objects where each of the objects represents some functionality. In this case, the objects in the system requires to use (work/collaborate with) some other objects to complete the given request. The objects with which an object collaborates to provide the service are known as its dependencies. In this context, we find a requirement of objects connecting to its dependencies (other objects) to get complete.
- That means one of the important concerns that we have in creating the enterprise application using OOP concepts is to connect the objects with its dependencies. We find the following three mechanisms doing this:
 - Dependency creation
 - Dependency pull
 - Dependency injection

Dependency Creation

- The dependency creation is the most commonly used mechanism of connecting an object with its dependency. In this case, the object takes the responsibility to create its dependency. For example

```

Class EmployeeServices{
    Private EmployeeDAO edao;
    Public EmployeeServices(){
        edao=new EmployeeDAO();
    }
}

```

Dependency Pull

- In the dependency pull approach, we access some source producing the objects (known as factory) to get the dependency for this object. The source such as a factory and naming service is accessed for getting the dependency. The dependency pull describes to have a factory to pull the dependency.

```
public class EmployeeServices{
    Private EmployeeDAOI edao;

    public EmployeeServices(){
        edao=DAOFactory.getEmployeeDAO();
    }
}
```

DI

- The dependency Injection is the process of injecting (pushing) the dependency into an object. This describes have the dependencies of the object get injected (pushed).
- This approach provides great simplification in creating the object type, as the object here will be free from creating or worried about getting the dependency.
- In this case, the object expects the client setting all the dependencies before or at the time of requesting for the service.

```
public class EmployeeServices{
    private EmployeeDAOI edao;
    public EmployeeServices(EmployeeDAOI edao){
        this.edao=edao;
    }
}
```

Spring Core Container

- It is an IOC Container implemented in Java for “Wiring Java Objects”.
- The Spring core containers primary job is creating the Java object by injecting its dependencies at creation time and manage them.
- The Spring Core Container can create any Java object, that is, supports creating POJO's. And the Java object created and managed by the spring core container is referred as “Spring Bean” i.e. is any Java object can be a Spring Bean.
- The spring core container supports injecting the primitives, array, collections and spring beans.

- The spring core container can inject the dependencies through the following ways.
 - Constructor injection
 - Setter injection.
- The `org.springframework.beans.factory.BeanFactory` interfaces provides the basic end point for the spring core container towards the applications to access the spring beans from the container.
- The `org.springframework.context.ApplicationContext` interface is the subtype of `BeanFactory` the provides some added functionalities like message resource handling, easy integration with AOP features and application layer etc.

Spring Core Container Main Responsibilities:

- Reads configuration
- Creates beans
- Injects dependencies
- Manages bean lifecycle
- Removes objects when the container closes

Types of Container

BeanFactory vs ApplicationContext in Spring

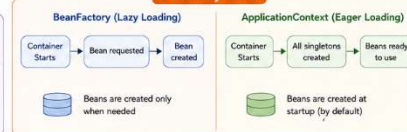
Both are Spring Containers and are responsible for managing Spring Beans.

1. BeanFactory			VS	2. ApplicationContext		
	Type	Basic container (Provides basic IoC support)			Type	Advanced container (Full-fledged enterprise container)
	Initialization	Lazy initialization (Beans are created only when requested)			Initialization	Eager initialization (by default) (Beans are created at startup)
	Weight	Lightweight (Consumes less memory and resources)			Additional Features	• Internationalization (i18n) support • Event handling • Bean post processors • AOP support • Validation, Environment support, etc.
	Use Case	Suitable for small applications where resource usage is critical			Use Case	Used in almost every Spring application (Web, Enterprise, REST APIs, etc.)
	Example Implementation	<code>XmlBeanFactory</code> (Deprecated), <code>DefaultListableBeanFactory</code>			Example Implementation	<code>ClassPathXmlApplicationContext</code> , <code>AnnotationConfigApplicationContext</code> , <code>WebApplicationContext</code>

Key Point

- `ApplicationContext` extends `BeanFactory`.
- So, `ApplicationContext` includes all features of `BeanFactory` plus many enterprise features.
- If you are building any real-world Spring application, `ApplicationContext` is the recommended choice.

How They Work



Quick Comparison

Feature	BeanFactory	ApplicationContext
Type	Basic	Advanced
Initialization	Lazy	Eager (by default)
Weight	Lightweight	Heavyweight
Features	Basic IoC	Enterprise features (i18n, events, AOP, etc.)
Use	Small applications	Almost all Spring apps

In Short:

Use **BeanFactory** when you need a lightweight container with basic features.
Use **ApplicationContext** for a full-featured, production-ready Spring application.



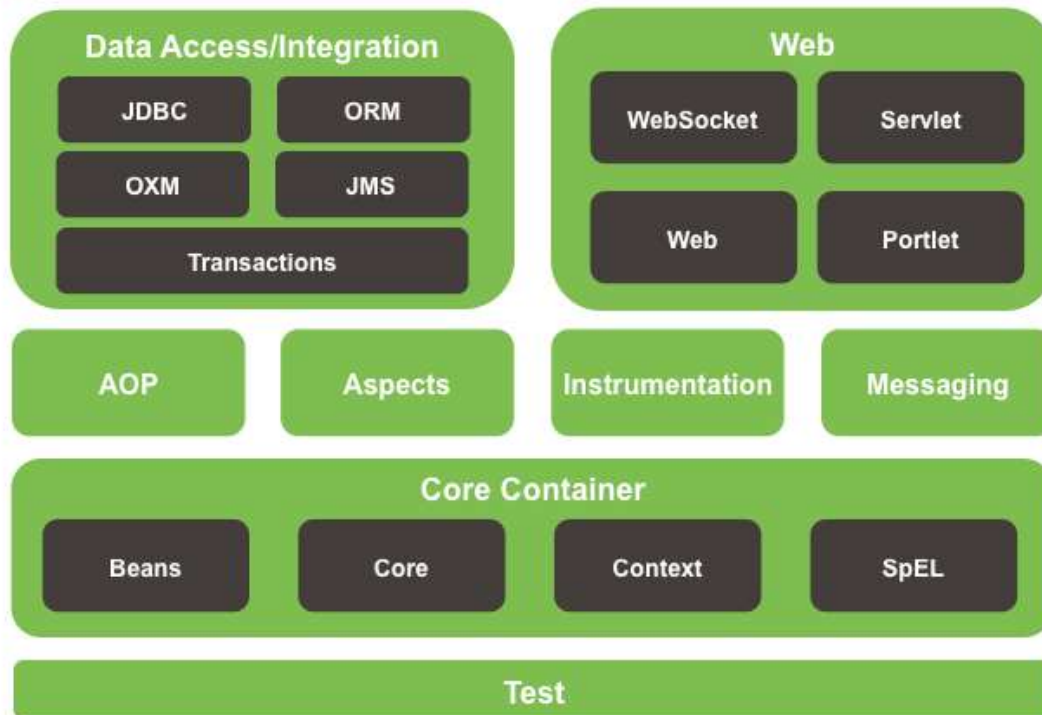
Best Practice:

In real-time projects, always prefer **ApplicationContext**.

Spring Modules



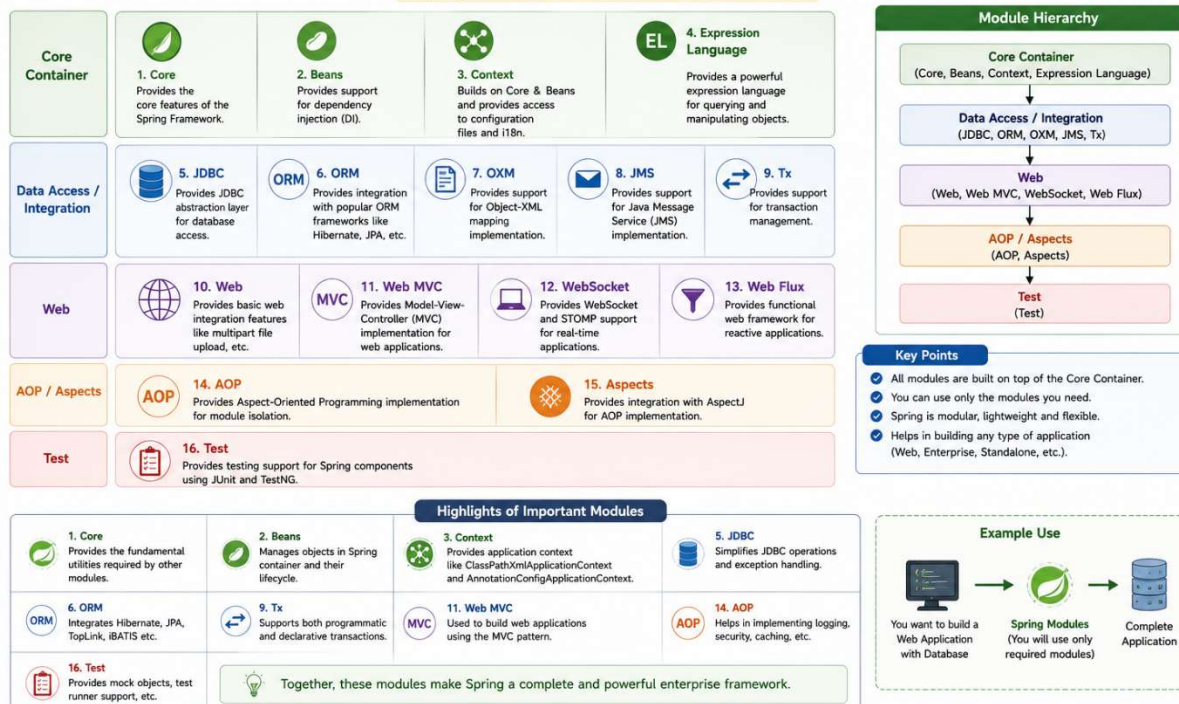
Spring Framework Runtime



Ref Link : <https://docs.spring.io/spring-framework/docs/4.0.x/spring-framework-reference/html/overview.html>

Spring Framework Modules

Spring Framework is divided into multiple modules.



Ref : <https://docs.spring.io/spring-framework/docs/4.0.x/spring-framework-reference/html/overview.html>

Exception Handling

Definition:

Exception Handling is a mechanism to handle runtime errors without crashing the application.

In Spring:

Spring provides built-in exception handling and converts database-specific exceptions into common exceptions, making code cleaner and easier to maintain.

Example:

Instead of showing a technical error, an ATM displays **"Insufficient Balance"**.

Integration with Other Frameworks

Definition:

Spring easily integrates with other Java technologies and frameworks.

Examples:

- Hibernate / JPA → Database
- Spring Boot → Rapid application development

- Angular / React → Frontend
- MySQL / Oracle → Database
- Spring Security → Authentication

Real-Time Example:

Angular (UI) → Spring Boot → Hibernate → MySQL

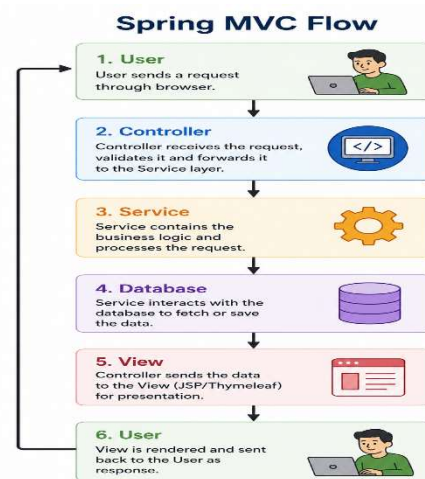
MVC (Model-View-Controller)

Definition:

MVC is a design pattern that separates an application into three layers.

- **Model** → Holds data (Employee, Student)
- **View** → Displays data (JSP, HTML, Thymeleaf)
- **Controller** → Receives requests and controls the flow

Real-Time Example:



Advantage:

Separates business logic from the user interface, making applications easier to develop and maintain.

ApplicationContext

Heading	Description	Example
Definition	ApplicationContext is an advanced IoC Container with enterprise features.	Most commonly used container in Spring.
Initialization	Eager Initialization	Creates singleton beans during startup.
Features	Event Handling, Internationalization, Annotation Support	Used in almost every Spring application.

```
ApplicationContext context =
    new ClassPathXmlApplicationContext("students.xml");

Student student = context.getBean("s", Student.class);

System.out.println(student);
```

ClassPathXmlApplicationContext

Heading	Description	Example
Definition	Loads XML configuration from the project's classpath.	Reads XML from src/main/resources.
Use	XML file inside the project	students.xml

Project Structure

```
src
├── main
│   └── resources
│       └── students.xml
```

```
ApplicationContext context =
    new ClassPathXmlApplicationContext("students.xml");
```

FileSystemXmlApplicationContext

Heading	Description	Example
Definition	Loads XML configuration from the file system.	Reads XML outside the project.
Use	External XML configuration	D:/Spring/students.xml

```
ApplicationContext context =
    new FileSystemXmlApplicationContext(
        "D:/Spring/students.xml");
```

AnnotationConfigApplicationContext

Heading	Description	Example
Definition	Loads bean definitions using Java Configuration instead of XML.	Uses @Configuration and @Bean.
Use	Annotation-based projects	Modern Spring applications

```
@Configuration
public class AppConfig {

    @Bean
    public Student student() {
        return new Student();
    }
}
```

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);

Student student = context.getBean(Student.class);
```

ConfigurableApplicationContext

Heading	Description	Example
Definition	Extension of ApplicationContext with lifecycle management methods.	Can close or refresh the container.
Methods	close(), refresh(), start(), stop()	Used to manage application context lifecycle.

DAY 2 – Morning Session (4 Hours)**Module: Spring Configuration & Dependency Injection****Topics**

1. Spring Configuration Metadata
2. XML Configuration
3. Java-based Configuration
4. Annotation-based Configuration
5. Dependency Injection (DI)
6. Types of DI
7. Constructor Injection
8. Setter Injection
9. Constructor vs Setter Injection
10. Field Injection
11. Collection Injection (List, Set, Map)

DAY 2 – Afternoon Session (4 Hours)**Module: Spring Beans & Bean Lifecycle****Topics**

1. Introduction to Beans
2. Bean Lifecycle
3. Initialization Callback
4. InitializingBean
5. init-method
6. @PostConstruct
7. Destruction Callback
8. DisposableBean
9. destroy-method
10. @PreDestroy
11. Bean Scopes
12. Singleton Scope
13. Prototype Scope
14. Shutting Down IoC Container

Day-2

Configuration Metadata

Spring Configuration Metadata is the information provided to the Spring IoC Container that tells it:

- Which objects (Beans) to create.
- How to configure them.
- How to inject dependencies.
- How beans are related to each other.

Why do we need Configuration Metadata?

Without configuration metadata, Spring does not know:

- Which class should become a bean.
- Which dependencies should be injected.
- Which scope to use.
- Which bean should be initialized first.

Example

Imagine you order food online.

The restaurant receives:

- Food Name
- Quantity
- Address
- Payment Method

This information is called **Order Details**.

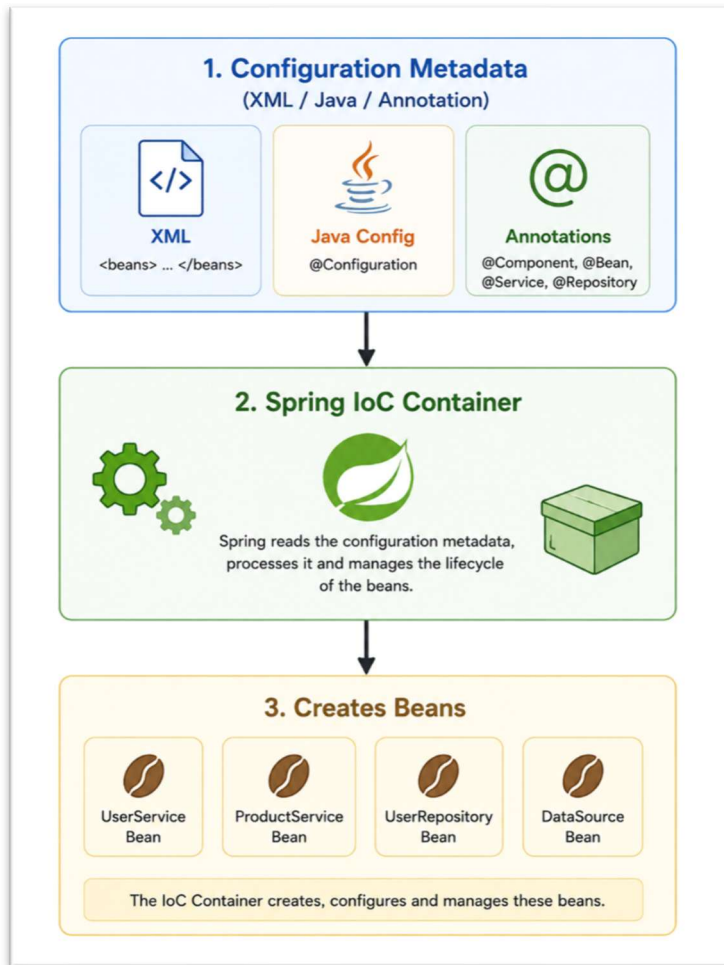
Similarly

Spring receives:

- Class Name
- Bean Id
- Dependencies
- Scope

This information is called **Configuration Metadata**.

Types of Configuration Metadata



Why XML Configuration?

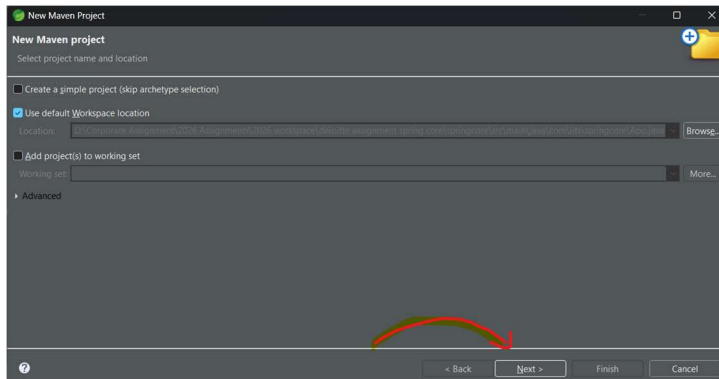
XML configuration keeps Java code separate from bean configuration.

Basic XML Structure

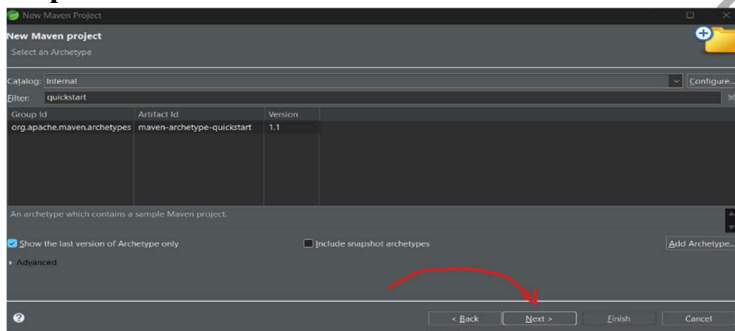
```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans">
  <bean id="student"
    class="com.iits.Student"/>
</beans>
```

Example1 : XML Based Configuration

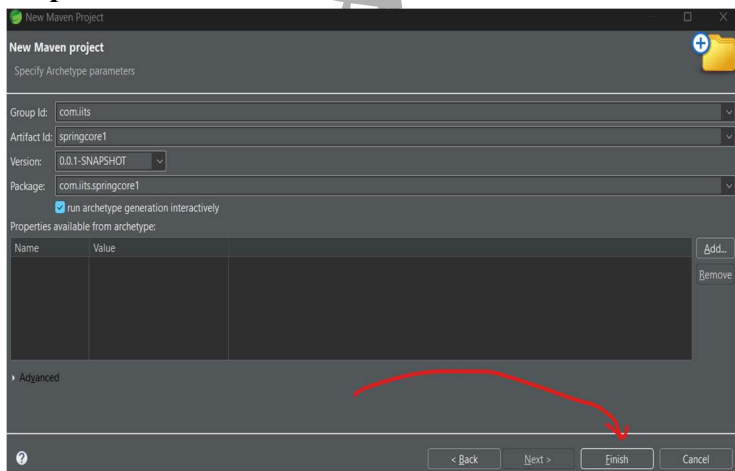
Step1: Create the maven project



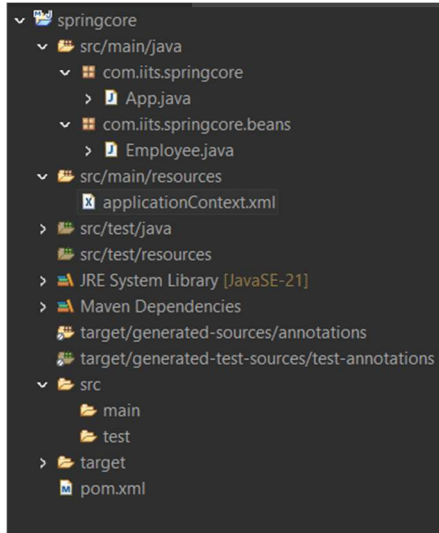
Step2:



Step3:



Step4: Project Structure



Step5: pom.xml file

```

2  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
3  <modelVersion>4.0.0</modelVersion>
4
5  <groupId>com.iits</groupId>
6  <artifactId>springcore</artifactId>
7  <version>0.0.1-SNAPSHOT</version>
8  <packaging>jar</packaging>
9
10 <name>springcore</name>
11 <url>http://maven.apache.org</url>
12
13<properties>
14  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15
16  <java.version>21</java.version>
17
18 </properties>
19
20<dependencies>
21  <!-- Source: https://mvnrepository.com/artifact/org.springframework/spring-core -->
22<dependency>
23  <groupId>org.springframework</groupId>
24  <artifactId>spring-core</artifactId>
25  <version>6.2.19</version>
26  <scope>compile</scope>
27 </dependency>
28 <!-- Source: https://mvnrepository.com/artifact/org.springframework/spring-context -->
29<dependency>
30  <groupId>org.springframework</groupId>
31  <artifactId>spring-context</artifactId>
32  <version>6.1.20</version>
33  <scope>compile</scope>
34 </dependency>
35<dependency>
36  <groupId>junit</groupId>
37  <artifactId>junit</artifactId>
38  <version>3.8.1</version>
39  <scope>test</scope>
40 </dependency>
41 </dependencies>
42 </project>

```

pom.xml (Spring Core Project)



Step6: Create the bean

Employee.class

```
package com.iits.springcore.beans;

public class Employee {

    public Employee() {
        System.out.println("---Employee Constructor ---");
    }

    public void message() {
        System.out.println("-Welcome to Employee Dashboard-");
    }
}
```


Disadvantages

- Large XML files
- Difficult to maintain
- Not preferred in modern Spring applications

Java-based Configuration

Instead of XML, configuration is written in a Java class.

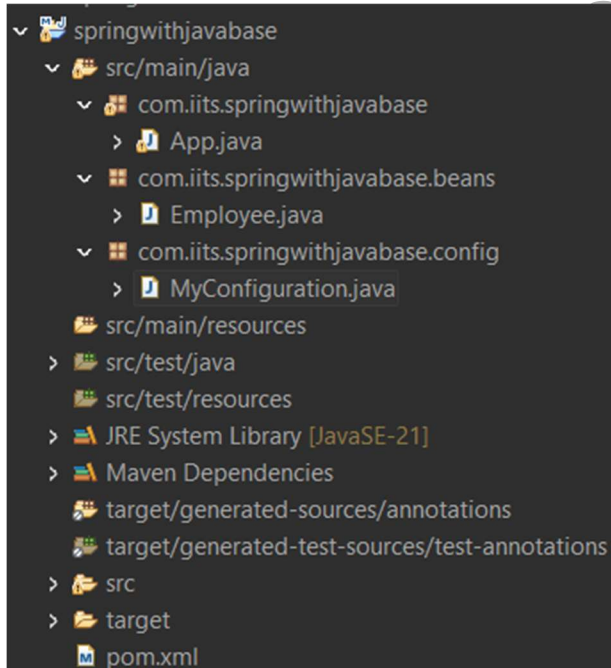
Spring uses:

- `@Configuration`
- `@Bean`

Example:

Step1 : Create the maven project is same as xml based.

Step2: Project Structure



Step3: pom.xml file same as xml based we have done above.

Step4: Create the MyConfiguraiton.java

```
package com.iits.springwithjavabase.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import com.iits.springwithjavabase.beans.Employee;

@Configuration
public class MyConfiguration {

    @Bean(name = "employee")
    public Employee createEmployee() {
        Employee emp=new Employee();
        return emp;
    }
}
```

Step5: App.java

```
package com.iits.springwithjavabase;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;

import com.iits.springwithjavabase.beans.Employee;
import com.iits.springwithjavabase.config.MyConfiguration;

public class App {
    public static void main(String[] args) {
        ApplicationContext context=
            new AnnotationConfigApplicationContext(MyConfiguration.class);

        Employee emp=
            context.getBean("employee",Employee.class);
        emp.message();

        ((AbstractApplicationContext) context).close();
    }
}
```

Advantages

- Type-safe
- No XML required
- Easy to maintain
- IDE support

IQ Test Your Skill

Q1: Which annotation is used to define a Java Configuration class?

Annotation-based Configuration

Here Instead of XML or Java configuration, Spring automatically detects beans using annotations.

Common Annotations:

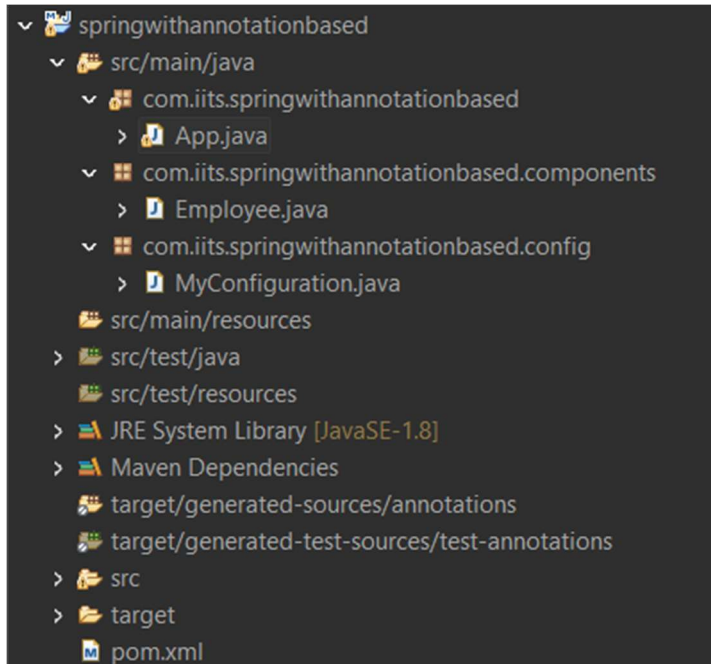
Annotation	Purpose
@Component	Generic Spring Bean
@Service	Service Layer
@Repository	DAO Layer
@Controller	Controller Layer
@Autowired	Inject Dependency

Example:

Step1: Create the maven project(quick start)

Step2: Add the spring core dependencies in pom.xml file

Step3: Project Structure



Step4: Create the component Employee.java

```
package com.iits.springwithannotationbased.components;

import org.springframework.stereotype.Component;

@Component
public class Employee {

    public Employee() {
        System.out.println("--Employee Object is created---");
    }

    public void message() {
        System.out.println("--Welcome to Dashboard---");
    }
}
```

Step5: Create the MyConfiguraiton.java

```

package com.iits.springwithannotationbased.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.iits.springwithannotationbased")
public class MyConfiguration {

}

```

Step6: Create the App.java

```

package com.iits.springwithannotationbased;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;

import com.iits.springwithannotationbased.components.Employee;
import com.iits.springwithannotationbased.config.MyConfiguration;

public class App {
    public static void main(String[] args) {
        ApplicationContext context
        = new AnnotationConfigApplicationContext(MyConfiguration.class);

        Employee employee=
            context.getBean(Employee.class);
        employee.message();

        ((AbstractApplicationContext) context).close();
    }
}

```

Dependency Injection

Dependency Injection (DI) is a design pattern in which the **Spring IoC Container** creates and injects the dependent objects into another object instead of the object creating them itself.

Dependency Injection means Spring creates the required object and injects it into another object.

Why do we need DI?

- Without DI, classes are tightly coupled.

Without DI

```
public class Employee {

    Address address = new Address();

}
```

What are the challenges we can face of above one?

- Employee creates Address.
- Tight Coupling
- Difficult to test
- Difficult to maintain

With DI

```
public class Employee {

    @Autowired
    private Address address;

}
```

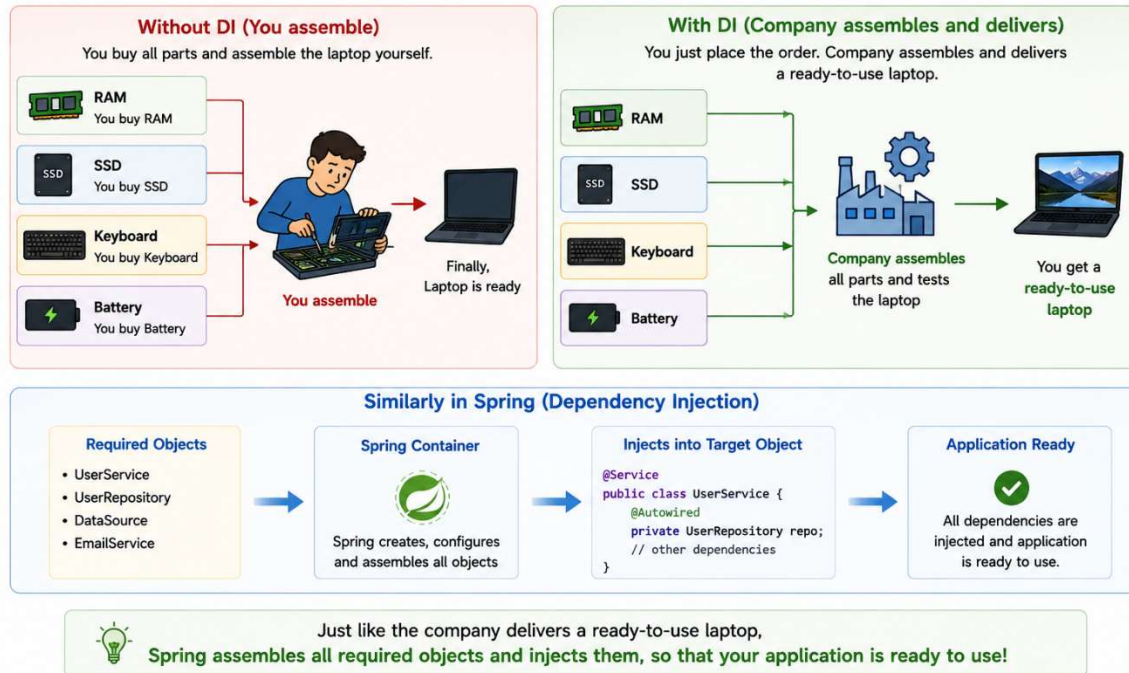
Note : Spring injects the Address object

Advantages

- Loose Coupling
- Easy Testing
- Better Maintainability
- Reusable Code

Real-Time Example

Let's understand Dependency Injection (DI) with a real-life example.



Types of DI

Types of Dependency Injection

Dependency Injection (DI) is the design pattern in which the IoC container provides the dependencies to a class instead of the class creating them itself.

Type	Description	Example	How it Works
1 Constructor Injection 	- Dependencies are provided through the constructor. - Ensures that the dependency is available before the object is used. Recommended way.	<pre>@Component public class UserService { private final UserRepository repo; @Autowired public UserService(UserRepository repo) { this.repo = repo; } }</pre>	Container creates UserRepository and injects it into UserService through the constructor.
2 Setter Injection 	- Dependencies are provided through setter method. - The object can be created without the dependency. - Useful when dependency is optional.	<pre>@Component public class UserService { private UserRepository repo; @Autowired public void setUserRepository(UserRepository repo) { this.repo = repo; } }</pre>	Container creates the object first and then calls the setter method to inject the dependency.
3 Field Injection 	- Dependencies are injected directly into the fields. - Simple to use but not recommended. - Makes unit testing difficult.	<pre>@Component public class UserService { @Autowired private UserRepository repo; }</pre>	Container injects the dependency directly into the field using reflection.

Note: Constructor Injection is the best practice because it ensures immutability, makes the required dependencies explicit, and helps in writing testable and maintainable code.

Injecting Collections in Spring: List, Set, and Map

Spring Dependency Injection can inject collections of values or objects into a bean. The commonly used collections are:

- **List** → Allows duplicates and maintains insertion order.
- **Set** → Does not allow duplicates.
- **Map** → Stores data as key-value pairs.

1. Injecting a List

XML Configuration

```
<bean id="employee" class="com.example.Employee">
  <property name="skills">
    <list>
      <value>Java</value>
      <value>Spring</value>
      <value>MySQL</value>
    </list>
  </property>
</bean>
```

```
public class Employee {

    private List<String> skills;

    public void setSkills(List<String> skills) {
        this.skills = skills;
    }

    public void display() {
        System.out.println(skills);
    }
}
```

2. Injecting a Set

A Set is useful when duplicate values should be avoided.

XML Configuration

```
<bean id="student" class="com.example.Student">
  <property name="courses">
    <set>
      <value>Java</value>
      <value>Spring</value>
    </set>
  </property>
</bean>
```

```

        <value>Java</value>
    </set>
</property>
</bean>

```

```

public class Student {

    private Set<String> courses;

    public void setCourses(Set<String> courses) {
        this.courses = courses;
    }

    public void display() {
        System.out.println(courses);
    }
}

```

3. Injecting a Map

A Map stores values using key–value pairs.

XML Configuration

```

<bean id="employee" class="com.example.Employee">
    <property name="departments">
        <map>
            <entry key="101" value="IT"/>
            <entry key="102" value="HR"/>
            <entry key="103" value="Finance"/>
        </map>
    </property>
</bean>

```

```

public class Employee {

    private Map<Integer, String> departments;

    public void setDepartments(Map<Integer, String> departments) {
        this.departments = departments;
    }

    public void display() {
        System.out.println(departments);
    }
}

```

Injecting Collections Using Java-Based Configuration

```
@Configuration
public class AppConfig {

    @Bean
    public Employee employee() {

        List<String> skills =
            Arrays.asList("Java", "Spring", "MySQL");

        return new Employee(skills);
    }
}
```

```
public class Employee {

    private List<String> skills;

    public Employee(List<String> skills) {
        this.skills = skills;
    }

    public void display() {
        System.out.println(skills);
    }
}
```

Set

```
@Bean
public Student student() {

    Set<String> courses =
        new HashSet<>(Arrays.asList(
            "Java", "Spring", "Java"
        ));

    return new Student(courses);
}
```

Map

```
@Bean
public Employee employee() {

    Map<Integer, String> departments = new HashMap<>();

    departments.put(101, "IT");
    departments.put(102, "HR");
    departments.put(103, "Finance");

    return new Employee(departments);
}
```

XML Based

```
<bean id="employee" class="Employee"/>
```

JavaBased

```
@Bean
public Employee employee() {
    return new Employee();
}
```

Note:

Java-based configuration is generally preferred in modern Spring applications because it is type-safe, refactoring-friendly, and easier to manage using Java code.

Spring Beans, Bean Life Cycle & Bean Scopes

Introduction

A **Spring Bean** is an object that is created, configured, and managed by the Spring IoC (Inversion of Control) Container.

Example;

```
@Component
public class Employee {
}
```

When component scanning is enabled, Spring creates and manages an object of *Employee*.

Example

@Configuration

```
public class AppConfig {
```

@Bean

```
public Employee employee() {
    return new Employee();
}
}
```

Spring Bean Life Cycle

The **Spring Bean Life Cycle** describes the complete journey of a Spring Bean: **from the moment Spring creates the bean until the moment Spring destroys the bean.**



Why Do We Need Bean Life Cycle?

Suppose your application has a bean that needs to:

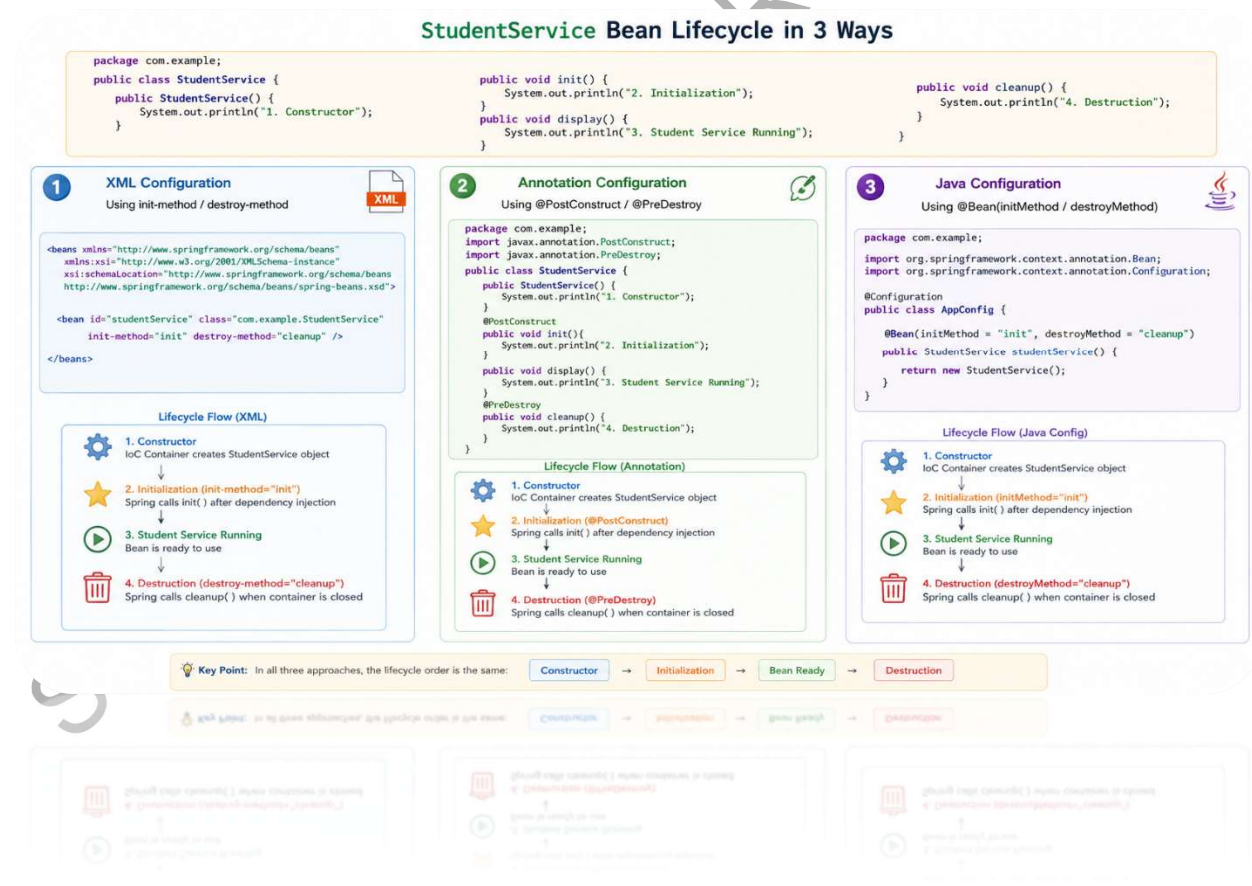
- Load some initial data
- Initialize a cache
- Open a resource
- Validate configuration
- Start a scheduler

You want these operations to happen **automatically when the bean is ready**.

Similarly, when the application shuts down, you may need to:

- Close a connection
- Stop a scheduler
- Release resources
- Flush data

Bean lifecycle callbacks allow us to tell Spring: "Execute this method when my bean is initialized."



Spring Bean Scope

- ✚ Bean Scope defines how Spring creates and manages instances of a bean.
- ✚ Bean Scope tells Spring how many objects of a particular bean should be created and how long those objects should live.

Let's think like this:

```
@Service  
public class StudentService {  
}
```

When different parts of the application ask Spring for StudentService, should Spring give them the same object or create a new object?

That is controlled by **Bean Scope**.

Types of Bean Scope in Spring

Bean Scope defines **how long a Bean lives** and **how many instances** Spring creates

1 Singleton Scope (Default)

- One instance per Spring IoC container
- Same object is returned every time
- Default scope in Spring

```
@Component
@Scope("singleton")
public class StudentService { }
```

2 Prototype Scope

- A new object is created every time
- Useful when each client needs its own bean instance

```
@Component
@Scope("prototype")
public class StudentService { }
```

3 Request Scope

- One bean instance is created for each HTTP request
- Mainly used in web applications

```
@Component
@Scope("request")
public class RequestData { }
```

4 Session Scope

- One bean instance is created for each HTTP session
- Same bean is used throughout that user's session

```
@Component
@Scope("session")
public class UserSession { }
```

5 Application Scope

- One bean instance is created for the ServletContext / application
- Shared across the entire web application

```
@Component
@Scope("application")
public class ApplicationData { }
```

6 WebSocket Scope

- One bean instance is created for each WebSocket session
- Used in applications with WebSocket communication

```
@Component
@Scope("websocket")
public class WebSocketData { }
```

Quick Comparison

Scope	Number of Instances	Lifetime	Where it is used
singleton	1 per IoC container	Container lifetime	All applications
prototype	New instance each request	Until client manages it	All applications
request	1 per HTTP request	HTTP request	Web applications
session	1 per HTTP session	HTTP session	Web applications
application	1 per ServletContext	Web application	Web applications
websocket	1 per WebSocket session	WebSocket session	Web applications

Easy Way to Remember

singleton	→	1 Container
prototype	→	1 getBean()
request	→	1 HTTP Request
session	→	1 HTTP Session
application	→	1 Web Application
websocket	→	1 WebSocket Session

Important: singleton and prototype are core Spring scopes. request, session, application and websocket are web-aware scopes and require an appropriate web application context.

DAY	2
TOPIC	https://github.com/imtiazdeloitetrainer/spring-day2-exercise.git
Exercise Link	